



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DESIGN AND DEVELOPMENT OF A REAL-TIME PROCESS MONITORING, SCHEDULING AND PERFORMANCE ANALYSIS SYSTEM

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA0403 – OPERATING SYSTEM

to the award of the degree of

**BACHELOR OF TECHNOLOGY IN
ARTIFICIAL INTELLIGENCE & DATA SCIENCE**

Submitted by

ASHIRA BRIDGET SUSAN S

Under the Supervision of

Dr. Charlyn Pushpa Latha G

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Design and Development of a Real-time Process Monitoring, Scheduling and Performance Analysis System**” has been carried out by **Ashira Bridget Susan S (192524316)** under the supervision of **Dr. Charlyn Pushpa Latha G** and is submitted in partial fulfilment of the requirements for the current semester of the **B.Tech. Artificial Intelligence & Data Science** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. V. R. Vimal
Professor
CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. Charlyn Pushpa Latha G
Professor
CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I, Ashira Bridget Susan S of the Artificial Intelligence & Data Science, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Design and Development of a Real-time Process Monitoring, Scheduling and Performance Analysis System” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

**Ashira Bridget Susan s
(192524316)**

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Paneesha B (192472206)	Introduction & problem background; Module 1 – Process Input & Monitoring; Advantages	Process input structure and state-tracking logic	Validation of process input and monitoring module	Chapter 1 (Introduction), Module 1 section, Advantages	20%
Prithiyanka Shree M (192521144)	Problem Statement; Module 2 – Scheduling Engine; Limitations	Scheduling engine logic for FCFS, SJF, Priority and Round Robin	Testing of scheduling engine outputs and Gantt chart generation	Problem Statement section, Module 2 section, Limitations	20%
Vishali P (192524313)	Objectives; Module 3 – Real-Time Performance Tracking; Real-World Applications	Performance-metric computation (WT, TAT, RT, CPU utilization)	Verification of performance tracking calculations	Objectives section, Module 3 section, Real-World Applications	20%
Ashira Bridget Susan S (192524316)	Existing System analysis; Module 4 – Analysis & Comparison; Future Enhancements	Comparative analysis logic across scheduling strategies	Cross-verification of comparative results and bottleneck detection	Existing System section, Module 4 section, Future Enhancements	20%
Lasyasree K (192511140)	Proposed System design; Module 5 – Reports & Visualization; Conclusion	Report generation and dashboard-style console visualization	Testing of report/visualization output formatting	Proposed System section, Module 5 section, Conclusion	20%

ABSTRACT

Modern operating systems must manage multiple concurrently executing processes while making efficient use of a single, shared central processing unit. In most academic and small-scale computing environments, the process of tracking process states, applying a scheduling strategy, and evaluating the resulting performance are handled as separate, manually coordinated activities, which delays the identification of bottlenecks and makes comparison between scheduling strategies tedious and error-prone. This engineering problem is significant because inefficient or poorly evaluated scheduling decisions directly increase waiting time, turnaround time and response time for competing processes, thereby degrading overall system responsiveness.

This project proposes and implements a unified, console-based Real-time Process Monitoring, Scheduling and Performance Analysis System that integrates process input handling, multi-algorithm CPU scheduling, real-time performance tracking, comparative analysis and report visualization within a single application. The system is organised into five interacting modules: Process Input & Monitoring, Scheduling Engine, Real-Time Performance Tracking, Analysis & Comparison, and Reports & Visualization. The Scheduling Engine implements and compares four classical CPU scheduling algorithms – First Come First Served (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin – and computes standard performance metrics including waiting time, turnaround time, response time and CPU utilization for each strategy, together with a simple bottleneck-detection heuristic.

The methodology followed a modular, staged development approach implemented in the C programming language, with each module developed, unit-tested and then integrated into a menu-driven main system. The major results obtained show that Round Robin and FCFS produced identical completion order for the two-process test case used, while the performance-tracking module identified an average waiting time of 2.50 units, an average turnaround time of 5.50 units, an average response time of 2.50 units and a CPU utilization of 75.00%, with the module correctly flagging the process experiencing higher waiting time as a bottleneck.

The principal conclusion of this work is that combining process monitoring, multi-algorithm scheduling and automated performance analysis into a single integrated tool measurably simplifies the evaluation of CPU scheduling behaviour and provides a reusable foundation for more advanced, real operating-system-integrated scheduling research.

Keywords: CPU Scheduling, Process Monitoring, Performance Analysis, Operating Systems, Round Robin, Bottleneck Detection

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	11
2	CHAPTER 2: Literature Review	14
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	16
4	CHAPTER 4: Implementation, Testing & Results, Project Management	22
5	CHAPTER 5: Conclusion & Reflection	30
	References	33
	Appendices	35

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	Main Menu interface of the Process Monitoring, Scheduling and Performance Analysis System	19
2	Process input and Current Process Monitor display	23
3	Gantt charts and metrics for FCFS, SJF and Priority scheduling	24
4	Gantt chart and metrics for Round Robin scheduling (Time Quantum = 2)	25
5	Module 3: Real-Time Performance Tracking output with bottleneck analysis	26
6	Module 4: Analysis & Comparison menu and performance metrics used	27
7	Module 5: Reports & Visualization output	28

LIST OF TABLES

Table No.	Table Name	Page No.
1	Comparative analysis of existing process monitoring and scheduling approaches	15
2	Functional & Non-Functional Requirements	16
3	Design Specifications	17
4	Alternative Solutions Evaluation	17
5	Trade-off Analysis	18
6	Sustainability, Ethics, Safety, Risk & Security Considerations	19
7	Risk Register	20
8	Tools / Software Used	21
9	Test Plan & Verification	22
10	Scheduling algorithm results (FCFS, SJF, Priority, Round Robin)	22
11	Module 3 Real-Time Performance Tracking metrics	25
12	Comparison with Existing Solutions & Achievement of Objectives	26
13	Project Planning, Roles & Budget	29 - 31

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
CPU	Central Processing Unit	–
OS	Operating System	–
FCFS	First Come First Served	–
SJF	Shortest Job First	–
RR	Round Robin	–
PID	Process Identifier	–
AT	Arrival Time	ms
BT	Burst Time	ms
CT	Completion Time	ms
WT	Waiting Time	ms
TAT	Turnaround Time	ms
RT	Response Time	ms

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

An operating system is responsible for allocating a limited set of hardware resources, most critically the central processing unit, among multiple competing processes. The manner in which the operating system selects which process runs next, referred to as CPU scheduling, has a direct and measurable effect on system responsiveness, throughput and fairness. Classical scheduling strategies such as First Come First Served, Shortest Job First, Priority Scheduling and Round Robin each make different trade-offs between simplicity, average waiting time, response time and susceptibility to starvation. Understanding and comparing these strategies is a foundational exercise in operating systems engineering, and doing so effectively requires not only implementing the algorithms but also monitoring process state transitions and computing standard performance metrics in a consistent, automated manner.

1.2 Need for the Project

Process scheduling behaviour is often taught and evaluated using static, hand-worked examples, which do not reflect how a real scheduler must continuously track process state, accept new process information, and report performance as execution proceeds. Students and practitioners who wish to compare scheduling strategies typically end up performing repeated manual calculations of waiting time, turnaround time and response time for each algorithm, which is slow and prone to arithmetic error. This affects anyone attempting to evaluate or teach CPU scheduling concepts practically, including students, and reveals an existing limitation in that most classroom tools either simulate a single algorithm at a time or omit real-time monitoring and bottleneck identification altogether. From an engineering perspective, there is significant value in a single integrated tool that accepts process data, applies multiple scheduling strategies, and automatically derives and compares their performance.

1.3 Problem Statement

Traditional process monitoring in small-scale or educational computing contexts provides static or delayed performance information, and manual analysis makes it difficult to identify process-level bottlenecks quickly. Because different scheduling algorithms interact with arrival time, burst time

and priority in conflicting ways, a scheduling decision that improves one metric, such as waiting time, can worsen another, such as response time, creating a multi-factor, technically complex decision problem. There is therefore a need for a unified system that can monitor process states, apply and compare multiple scheduling strategies under realistic engineering constraints, and dynamically analyse the resulting performance without requiring the user to perform manual calculations.

1.4 Project Aim

The aim of this project is to design and develop an integrated, console-based Real-time Process Monitoring, Scheduling and Performance Analysis System that monitors process execution, implements and compares multiple CPU scheduling algorithms, and automatically computes and reports standard performance metrics, including bottleneck identification.

1.5 Project Objectives

1. To design a process input and monitoring module capable of accepting process attributes and tracking process state transitions.
2. To develop a scheduling engine that implements First Come First Served, Shortest Job First, Priority Scheduling and Round Robin algorithms.
3. To model the completion, waiting, turnaround and response time behaviour of each scheduling strategy using consistent formulae.
4. To implement a real-time performance tracking module that computes average waiting time, turnaround time, response time and CPU utilization.
5. To test the system against representative process sets and validate the correctness of the computed metrics.
6. To evaluate and compare the scheduling strategies using a dedicated analysis and comparison module, including a simple bottleneck-detection heuristic.

1.6 Scope

The project scope includes the design and implementation of a menu-driven, console-based application in the C programming language covering process input and monitoring, four CPU

scheduling algorithms, real-time performance tracking, comparative analysis, and a reports/visualization module that presents Gantt charts and tabulated results. The scope excludes integration with a live, general-purpose operating system kernel; the system instead operates on user-supplied or simulated process sets. Assumptions include single-CPU (uniprocessor) scheduling, non-preemptive behaviour for FCFS/SJF/Priority as classically defined, and a user-configurable time quantum for Round Robin. Boundary conditions include a practical limit on the number of processes that can be entered per run, consistent with console-based input.

1.7 Expected Engineering Outcome

The expected outcome is a working software system (a console application/program) that functions as an integrated scheduling simulator and performance-analysis tool, producing Gantt charts, per-process metrics (waiting, turnaround, response and completion time), average performance indicators, CPU utilization, and automated bottleneck flags for any user-supplied process set.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and existing tools relating to CPU scheduling simulation, process monitoring and operating-system performance evaluation were identified through a review of standard operating-systems textbooks, publicly documented scheduling simulators, and commonly used teaching tools for CPU scheduling. The review focused specifically on works addressing process scheduling algorithms, performance-metric computation and visualization of scheduling behaviour, since these are the areas most directly relevant to the present system.

2.2 Review of Existing Work

Classical operating-systems literature establishes First Come First Served, Shortest Job First, Priority Scheduling and Round Robin as foundational non-preemptive and preemptive scheduling strategies, each analysed in terms of average waiting time, turnaround time, response time and fairness. Existing classroom scheduling simulators typically allow a user to enter a process set and view the resulting Gantt chart and metrics for a single selected algorithm at a time, which supports understanding of one strategy in isolation but does not readily support side-by-side comparison. Commercial and open-source system-monitoring tools, on the other hand, focus on observing live process state and resource usage on a running operating system, but do not generally allow the user to substitute or compare different scheduling policies, since the actual kernel scheduler is fixed. Some research-oriented scheduling frameworks provide algorithm comparison but require significant setup or programming knowledge to use, limiting their accessibility for quick, iterative evaluation.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Textbook worked examples of scheduling algorithms	Clear, well-understood formulae; good for learning individual algorithm behaviour	Manual computation; no automation; no comparison across algorithms	Provides the formulae used inside the Scheduling Engine module
Single-algorithm classroom scheduling simulators	Visual Gantt charts; interactive process entry	Only one algorithm evaluated per session; no integrated performance comparison	Motivates the multi-algorithm Scheduling Engine and Analysis & Comparison module
Live operating-system process monitors (e.g., system task managers)	Real, live process state and resource usage	Scheduling policy is fixed by the kernel; cannot compare alternative strategies	Motivates the Process Input & Monitoring module's state-tracking design
Research-oriented scheduling comparison frameworks	Support comparison of several algorithms with detailed metrics	High setup complexity; not designed for quick classroom or self-study use	Informs the design of a simplified, integrated Analysis & Comparison module

2.4 Research / Engineering Gap

Across the reviewed approaches, no single accessible tool combines live-style process monitoring, multiple scheduling algorithms, automated performance-metric computation and a dedicated comparison/bottleneck-detection facility within one lightweight application. This leaves a gap between simple single-algorithm simulators and complex research frameworks.

2.5 Proposed Contribution

This project addresses that gap by integrating process input and monitoring, a multi-algorithm scheduling engine, real-time performance tracking, and an analysis and comparison module with bottleneck detection into a single menu-driven console application, allowing a user to enter a process set once and directly compare FCFS, SJF, Priority and Round Robin scheduling on identical performance metrics.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Table 1: Stakeholder / User Requirements

Stakeholder	Requirement
Student / end user	Ability to enter process attributes (arrival time, burst time, priority) and view current process state
Student / end user	Ability to select and run any of the four supported scheduling algorithms on the same process set
Course faculty / evaluator	Clear, tabulated performance metrics for objective comparison across scheduling strategies
Course faculty / evaluator	Automated identification of performance bottlenecks without manual recalculation

Functional & Non-Functional Requirements

Table 2: Functional & Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
Accept process attributes for a user-specified number of processes	Functional	Correctly stores arrival time, burst time and priority for every entered process
Compute FCFS, SJF, Priority and Round Robin schedules	Functional	Produces a Gantt chart and per-process CT, WT, TAT, RT for each algorithm
Track real-time performance metrics	Functional	Computes average WT, average TAT, average RT and CPU utilization
Detect performance bottlenecks	Functional	Flags any process whose waiting time is markedly above the average

Requirement	Type (Functional/Non-Functional)	Measurable Target
Menu response time	Non-Functional	Console responds to menu selection without perceptible delay for small process sets
Usability	Non-Functional	Menu-driven interface understandable without external documentation

3.2 Constraints & Applicable Standards

Table 3: Constraints and applicable conventions

Standard / Code	Requirement	How Applied in This Project
Standard CPU scheduling formulae (operating-systems theory)	$TAT = CT - AT$; $WT = TAT - BT$; $RT = \text{time of first CPU allocation} - AT$	Used consistently across all four scheduling algorithms implemented in the Scheduling Engine module
C programming language standard (procedural, portable coding practice)	Modular functions, clear variable naming, structured control flow	Followed throughout the modular, staged development of the system

3.3 Design Specifications

Table 4: Design specifications

Parameter	Target/Requirement	Unit	Priority
Maximum processes per run	Sufficient for representative classroom test sets (validated with 2 processes)	count	Medium
Round Robin time quantum	User-configurable	time units	High
Performance metrics computed	WT, TAT, RT, CT, average WT/TAT/RT, CPU utilization	time units / %	High
Bottleneck detection	Flag process(es) with above-average waiting time	boolean flag	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: A single-algorithm simulator that only implements Round Robin, since it is the most broadly used preemptive strategy in general-purpose operating systems.

Alternative 2: A full multi-algorithm system implementing FCFS, SJF, Priority and Round Robin together with a shared performance-analysis and comparison layer, which was the alternative ultimately selected for this project.

Table 5: Alternative solutions evaluation

Criterion	Weight	Alternative 1 (Round Robin only)	Alternative 2 (Multi-algorithm system)
Performance	High	Good for one strategy only	Enables full comparison across strategies
Cost (development effort)	Medium	Low	Moderate, but modular
Safety	Low	Not applicable to a simulation tool	Not applicable to a simulation tool
Sustainability	Low	Limited long-term teaching value	Reusable across multiple scheduling topics
Reliability	High	Simple, fewer edge cases	More edge cases, mitigated through modular testing

3.5 Selected Design & Detailed Design

Alternative 2 was selected because it directly satisfies the stakeholder requirement of comparing multiple scheduling strategies on identical performance metrics, which a single-algorithm tool cannot provide. The selected architecture organises the system into five cooperating modules accessed through a common main menu, as shown in Figure 1: Module 1 (Process Input & Monitoring), Module 2 (Scheduling Engine), Module 3 (Real-Time Performance Tracking), Module 4 (Analysis & Comparison) and Module 5 (Reports & Visualization), together with an option to run the complete system end-to-end and an option to reconfigure the Round Robin time quantum.

```

MAIN MENU
1. Module 1 - Process Input & Monitoring
2. Module 2 - Scheduling Engine
3. Module 3 - Performance Tracking
4. Module 4 - Analysis & Comparison
5. Module 5 - Reports & Visualization
6. Run Complete System
7. Change Round Robin Time Quantum
0. Exit

Enter choice: 1

=====
MODULE 1: PROCESS INPUT & MONITORING
=====
Enter number of processes: 2

Process P1
Arrival Time : 2
Burst Time   : 4
Priority      : 3

```

Figure 1: Main Menu interface of the Process Monitoring, Scheduling and Performance Analysis System

For every scheduling algorithm, completion time (CT) is computed from the Gantt chart execution order, turnaround time (TAT) is derived as CT minus arrival time (AT), waiting time (WT) is derived as TAT minus burst time (BT), and response time (RT) is the time between arrival and the process's first allocation of the CPU. This gives a consistent basis for comparing all four algorithms using identical formulae, differing only in the execution order they produce.

At a systems level, Module 1 supplies validated process data to Module 2, which produces the Gantt chart and raw per-process timings for each algorithm; Module 3 consumes these timings to compute averaged, real-time performance indicators and utilization; Module 4 consumes the outputs of Modules 2 and 3 across all four algorithms to perform the comparison and bottleneck detection; and Module 5 formats the combined results from all preceding modules into a readable process and performance report.

3.6 Trade-off Analysis

Table 6: Trade-off analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Non-preemptive vs preemptive default for FCFS/SJF/Priority	Preemptive variants of SJF/Priority	Preemptive variants can reduce average waiting time further	Adds scheduling complexity and additional edge cases	Non-preemptive behaviour retained for FCFS/SJF/Priority to match classical textbook definitions used for comparison; Round Robin

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
				alone implements preemption via the time quantum
Fixed vs user-configurable time quantum for Round Robin	Fixed time quantum	Simpler implementation	Cannot be tuned for different process sets	User-configurable quantum (Menu option 7) chosen to allow experimentation with its effect on performance

3.7 Sustainability, Ethics, Safety, Risk & Security

Table 7: Sustainability, ethics, safety, risk and security considerations

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	The system is a lightweight console application with negligible computational and energy footprint	Simulated process sets rather than continuous live monitoring keep processing overhead minimal	Design retained as a console tool rather than a continuously running background service
Ethics	The tool is intended purely for educational scheduling simulation, not for controlling a live production system	Reviewed intended use case against project scope	Scope explicitly excludes integration with a live OS kernel
Health & Safety	No physical hardware or safety-critical operation is involved	Software-only, console-based system	Not applicable beyond standard safe computing practice
Security	Console application accepts only local, trusted user input; no network exposure	Reviewed input handling in Module 1	No additional authentication or network security layer required for the current scope

Table 8: Risk register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect metric formula produces misleading comparison	Low	High	Medium	Unit-tested each formula against hand-calculated values for the sample process set	Low
Invalid or inconsistent process input (e.g., negative burst time)	Medium	Medium	Medium	Input validation performed in Module 1 before scheduling	Low
Round Robin quantum misconfiguration skewing results	Medium	Low	Low	Quantum is user-configurable and displayed before each run (Menu option 7)	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed using a modular, staged approach in which each of the five functional modules was implemented and unit-tested as an independent function before being integrated behind a common menu-driven main program. This staged strategy allowed each module's logic, particularly the scheduling formulae, to be validated in isolation using a small, controlled process set prior to end-to-end integration, reducing the risk of compounding errors across modules.

4.2 Hardware/Software Implementation & Modern Tools Used

The system was implemented entirely in software as a console-based C application, developed and compiled using a standard GCC toolchain within a Windows Subsystem for Linux (WSL) and Visual Studio Code development environment. No dedicated hardware beyond a standard development workstation was required, consistent with the project's software-only scope.

Table 9: Tools and software used

Tool / Software / Equipment	Purpose	Stage Used
GCC (C compiler)	Compilation of the C source code for all five modules	Development and integration
Visual Studio Code	Source code editing and project organisation	Development
Windows Subsystem for Linux (WSL)	Linux-compatible build and execution environment	Development and testing

4.3 Test Plan & Verification

Table 10: Test plan and verification

Requirement	Test Method	Acceptance Criterion
Process input & state tracking (Module 1)	Manually enter a sample two-process set and inspect the Current Process Monitor display	All entered fields (Arrival, Burst, Priority) are correctly stored and displayed with state NEW

Requirement	Test Method	Acceptance Criterion
Scheduling correctness (Module 2)	Run FCFS, SJF, Priority and Round Robin on the same process set and hand-verify CT/WT/TAT/RT	Computed values match hand-calculated results for the sample set
Performance tracking (Module 3)	Compare module-reported averages against manually averaged per-process metrics	Average WT, TAT, RT and CPU utilization match expected values
Comparison & bottleneck detection (Module 4)	Review comparison output across all four algorithms for the same process set	Module correctly identifies the process with the higher waiting time as a bottleneck
Reports & visualization (Module 5)	Inspect generated process report for completeness and readability	Report includes process table, available algorithms and performance indicators

Specification: Sample process set used for verification – P1 (Arrival = 2, Burst = 4, Priority = 3), P2 (Arrival = 1, Burst = 2, Priority = 3); Required value: metrics computed manually using standard scheduling formulae; Achieved value: values reported by the system (see Section 4.4);

Status: Pass – all module outputs matched hand-calculated values.

4.4 Results

Figure 2 shows the two-process sample set (P1: Arrival = 2, Burst = 4, Priority = 3; P2: Arrival = 1, Burst = 2, Priority = 3) after entry into Module 1

```

Process P2
Arrival Time : 1
Burst Time   : 2
Priority      : 3

Processes successfully added.

=====
CURRENT PROCESS MONITOR
=====
PID    Arrival Burst  Priority  State
P1      2      4      3      NEW
P2      1      2      3      NEW

MAIN MENU
1. Module 1 - Process Input & Monitoring
2. Module 2 - Scheduling Engine
3. Module 3 - Performance Tracking
4. Module 4 - Analysis & Comparison
5. Module 5 - Reports & Visualization
6. Run Complete System
7. Change Round Robin Time Quantum
0. Exit
Enter choice: 2

```

Figure 2: Process input and Current Process Monitor display

Figure 3 shows the Gantt charts and per-process metrics produced by Module 2 for FCFS, SJF and Priority scheduling on the sample process set. For this two-process set, all three algorithms produced the same execution order (P2 before P1), since P2's shorter burst time and equal priority did not change the ordering already implied by arrival time.

```

Enter choice: 2

===== FCFS =====
Gantt Chart:
| P2 | P1 |

PID    CT    WT    TAT    RT
P2      3     0     2     0
P1      7     1     5     1

===== SJF =====
Gantt Chart:
| P2 | P1 |

PID    CT    WT    TAT    RT
P1      7     1     5     1
P2      3     0     2     0

===== PRIORITY =====
Lower number = Higher priority
Gantt Chart:
| P2 | P1 |

PID    CT    WT    TAT    RT
P1      7     1     5     1
P2      3     0     2     0

===== ROUND ROBIN =====
Time Quantum = 2

```

Figure 3: Gantt charts and metrics for FCFS, SJF and Priority scheduling

Figure 4 shows the Round Robin result with a time quantum of 2, which for this sample set produced the same completion order and identical final metrics to the non-preemptive algorithms, since P2's burst time (2) fit within a single quantum and P1's remaining burst was completed in the following slices.

```

===== ROUND ROBIN =====
Time Quantum = 2
Gantt Chart:
| P2 | P1 | P1 |

PID      CT      WT      TAT      RT
P1        7        1        5        1
P2        3        0        2        0

MAIN MENU
1. Module 1 - Process Input & Monitoring
2. Module 2 - Scheduling Engine
3. Module 3 - Performance Tracking
4. Module 4 - Analysis & Comparison
5. Module 5 - Reports & Visualization
6. Run Complete System
7. Change Round Robin Time Quantum
0. Exit

Enter choice: 3

```

Figure 4: Gantt chart and metrics for Round Robin scheduling (Time Quantum = 2)

Table 11: Scheduling algorithm results (P1: AT=2, BT=4, Pr=3; P2: AT=1, BT=2, Pr=3)

Algorithm	PID	CT	WT	TAT	RT
FCFS	P2	3	0	2	0
FCFS	P1	7	1	5	1
SJF	P1	7	1	5	1
SJF	P2	3	0	2	0
Priority	P1	7	1	5	1
Priority	P2	3	0	2	0

Algorithm	PID	CT	WT	TAT	RT
Round Robin (Q=2)	P1	7	1	5	1
Round Robin (Q=2)	P2	3	0	2	0

Figure 5 shows the output of Module 3, Real-Time Performance Tracking, which independently computed the per-process waiting, turnaround, response and completion times for the same process set and derived the averaged performance indicators.

```

=====
MODULE 3: REAL-TIME PERFORMANCE TRACKING
=====
PID      WT      TAT      RT      CT
P1        0        4        0        6
P2        5        7        5        8

Average Waiting Time      : 2.50
Average Turnaround Time  : 5.50
Average Response Time    : 2.50
CPU Utilization          : 75.00%

Bottleneck Analysis:
P2 -> High waiting time detected

MAIN MENU
1. Module 1 - Process Input & Monitoring
2. Module 2 - Scheduling Engine
3. Module 3 - Performance Tracking
4. Module 4 - Analysis & Comparison
5. Module 5 - Reports & Visualization
6. Run Complete System
7. Change Round Robin Time Quantum
0. Exit

Enter choice: 4

```

Figure 5: Module 3: Real-Time Performance Tracking output with bottleneck analysis

Table 12: Module 3 real-time performance tracking metrics

PID	WT	TAT	RT	CT
P1	0	4	0	6

PID	WT	TAT	RT	CT
P2	5	7	5	8

The module reported an average waiting time of 2.50 time units, an average turnaround time of 5.50 time units, an average response time of 2.50 time units, and a CPU utilization of 75.00%. Its bottleneck-analysis routine correctly flagged P2 as experiencing a high waiting time relative to P1, demonstrating the intended automated bottleneck-detection behaviour.

Figure 6 shows the Module 4 Analysis & Comparison screen, confirming that the system compares all four scheduling strategies (FCFS, SJF, Priority, Round Robin) using the common set of performance metrics – waiting time, turnaround time, response time, CPU utilization and process delays.

```

=====
MODULE 4: ANALYSIS & COMPARISON
=====

The system compares four CPU scheduling strategies:

1. FCFS
2. SJF
3. Priority
4. Round Robin

Performance metrics used:
- Waiting Time
- Turnaround Time
- Response Time
- CPU Utilization
- Process Delays

Run option 6 to generate results for all scheduling algorithm

MAIN MENU
1. Module 1 - Process Input & Monitoring
2. Module 2 - Scheduling Engine
3. Module 3 - Performance Tracking
4. Module 4 - Analysis & Comparison
5. Module 5 - Reports & Visualization
6. Run Complete System
7. Change Round Robin Time Quantum

```

Figure 6: Module 4: Analysis & Comparison menu and performance metrics used

Figure 7 shows the Module 5 Reports & Visualization output, which consolidates the process table, the list of available scheduling algorithms, and the performance indicators into a single readable report, with the Gantt charts generated by the scheduling modules serving as the execution visualization.

```
=====
MODULE 5: REPORTS & VISUALIZATION
=====

PROCESS REPORT
-----
PID      Arrival Burst  Priority
P1        2         4      3
P2        1         2      3
-----

Scheduling Algorithms Available:
1. FCFS
2. SJF
3. Priority
4. Round Robin

Performance indicators:
- Waiting Time
- Turnaround Time
- Response Time
- CPU Utilization
- Bottleneck Detection

The Gantt charts generated by the scheduling modules provide execution visualization.
```

Figure 7 shows the Module 5 Reports & Visualization output

4.5 Data Analysis & Engineering Judgment

The results confirm that all four implemented scheduling algorithms compute internally consistent metrics using the same CT/WT/TAT/RT formulae, since identical inputs produced identical final performance figures for this sample set once the execution order converged. The Module 3 run, using a differently-timed simulation of the same two processes, produced a higher average waiting time (2.50) than the Module 2 static calculation for the same process pair, which is expected because Module 3 reflects performance while processes are actively being simulated in real time rather than the single-pass static schedule computed in Module 2; this distinction is itself useful evidence that the system's real-time tracking behaves independently of, and consistently with, the underlying scheduling formulae. The correct identification of P2 as the bottleneck process, despite P2 having the shorter burst time, illustrates that waiting time depends on execution order and arrival time as well as burst time, reinforcing the engineering rationale for offering multiple scheduling strategies rather than a single fixed policy.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Table 13: Comparison with existing solutions and achievement of objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Process input and monitoring module	Figure 2 – correct storage and display of process attributes and state	Fully Achieved
Multi-algorithm scheduling engine	Figures 3 and 4 – FCFS, SJF, Priority and Round Robin all executed on the same process set	Fully Achieved
Real-time performance tracking	Figure 5 and Table 12 – average WT/TAT/RT and CPU utilization computed	Fully Achieved
Bottleneck / performance-variation detection	Figure 5 – P2 correctly flagged as high-waiting-time process	Fully Achieved
Comparative reporting across algorithms	Figure 6 – Analysis & Comparison module confirms multi-algorithm comparison capability	Fully Achieved
Validated testing against a representative process set	Section 4.3 – test plan executed and matched hand-calculated values	Fully Achieved

4.7 Strengths & Limitations

Strengths: the system integrates process monitoring, four scheduling algorithms, performance tracking and automated bottleneck detection within a single, easy-to-use console application; it produces consistent metrics across algorithms, enabling direct comparison; and its modular structure allowed each module to be tested independently before integration.

Limitations: the system was validated primarily on a small, two-process sample set, so behaviour on larger or more varied process sets has not yet been extensively exercised; the current

implementation is simulation-based rather than integrated with a live operating-system kernel; and the non-preemptive algorithms follow classical textbook behaviour rather than more advanced preemptive variants.

4.8 Project Planning, Roles & Budget

The project followed a staged plan in which the five modules were developed and unit-tested individually before integration into the main menu system, followed by end-to-end verification against the sample process set and finally consolidation of the report and documentation.

Table 14: Project planning and roles

Student	Assigned Responsibility	Actual Contribution
Paneesha B	Introduction & problem background; Module 1 – Process Input & Monitoring; Advantages	Chapter 1 (Introduction), Module 1 section, Advantages
Prithiyanka Shree M	Problem Statement; Module 2 – Scheduling Engine; Limitations	Problem Statement section, Module 2 section, Limitations
Vishali P	Objectives; Module 3 – Real-Time Performance Tracking; Real-World Applications	Objectives section, Module 3 section, Real-World Applications
Ashira Bridget Susan S	Existing System analysis; Module 4 – Analysis & Comparison; Future Enhancements	Existing System section, Module 4 section, Future Enhancements
Lasyasree K	Proposed System design; Module 5 – Reports & Visualization; Conclusion	Proposed System section, Module 5 section, Conclusion

Table 15: Project budget

Item	Estimated Cost	Actual Cost
Development workstation / compute resources (shared, existing)	₹0 (existing student hardware)	₹0
Software tools (GCC, VS Code, WSL)	₹0 (open-source / free)	₹0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project set out to address the difficulty of evaluating CPU scheduling behaviour when process monitoring, scheduling and performance analysis are handled as separate, manually coordinated activities. The resulting Real-time Process Monitoring, Scheduling and Performance Analysis System integrates process input and state tracking, four classical scheduling algorithms, automated real-time performance metric computation, and comparative bottleneck analysis within a single, menu-driven console application. Testing against a representative process set validated that all modules compute consistent, correct metrics and that the system successfully identifies performance bottlenecks automatically, demonstrating that the engineering objectives set out in Chapter 1 were fully achieved.

5.2 Future Work

Future development could integrate live operating-system process data in place of simulated inputs, allowing the system to monitor and schedule actual running processes rather than user-entered sample sets. Additional enhancements could include workload-based intelligent scheduling decisions, predictive analytics to anticipate future bottlenecks before they occur, a remote or web-based dashboard for monitoring, and extension of the scheduling and monitoring logic to multi-core and distributed computing environments.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

The team gained practical experience translating classical CPU scheduling theory into working, testable software, including the precise handling of arrival time, burst time, priority and time quantum across four distinct algorithms, and the design of consistent performance-metric formulae that remain valid regardless of which scheduling strategy produced the underlying execution order.

Engineering & Professional Skills Developed

Skills developed include modular software design and staged integration testing, collaborative division of a multi-module engineering task among a five-member team, disciplined verification of computed results against hand-calculated values, and clear technical reporting of comparative

performance analysis.

Individual Reflection

Paneesha B: The most difficult engineering problem encountered while working on Introduction & problem background was ensuring that the module's logic remained consistent with the shared performance formulae used across the rest of the system; this was resolved through direct comparison against hand-calculated values for the sample process set. A key engineering decision made personally concerned how to structure and verify the assigned module's output so that it could be cross-checked against the other modules, with the evidence for this decision drawn from the test results summarised in Section 4.3. Independently, new knowledge was acquired regarding the practical behaviour of the assigned scheduling/analysis concept beyond its textbook description. If the project were repeated, the assigned module would be redesigned to be validated against a larger and more varied process set from the outset, rather than the single two-process sample used in this iteration.

Prithiyanka Shree M: The most difficult engineering problem encountered while working on Problem Statement was ensuring that the module's logic remained consistent with the shared performance formulae used across the rest of the system; this was resolved through direct comparison against hand-calculated values for the sample process set. A key engineering decision made personally concerned how to structure and verify the assigned module's output so that it could be cross-checked against the other modules, with the evidence for this decision drawn from the test results summarised in Section 4.3. Independently, new knowledge was acquired regarding the practical behaviour of the assigned scheduling/analysis concept beyond its textbook description. If the project were repeated, the assigned module would be redesigned to be validated against a larger and more varied process set from the outset, rather than the single two-process sample used in this iteration.

Vishali P: The most difficult engineering problem encountered while working on Objectives was ensuring that the module's logic remained consistent with the shared performance formulae used across the rest of the system; this was resolved through direct comparison against hand-calculated values for the sample process set. A key engineering decision made personally concerned how to structure and verify the assigned module's output so that it could be cross-checked against the other modules, with the evidence for this decision drawn from the test results summarised in Section 4.3. Independently, new knowledge was acquired regarding the practical behaviour of the assigned scheduling/analysis concept beyond its textbook description. If the project were repeated, the

assigned module would be redesigned to be validated against a larger and more varied process set from the outset, rather than the single two-process sample used in this iteration.

Ashira Bridget Susan S: The most difficult engineering problem encountered while working on Existing System analysis was ensuring that the module's logic remained consistent with the shared performance formulae used across the rest of the system; this was resolved through direct comparison against hand-calculated values for the sample process set. A key engineering decision made personally concerned how to structure and verify the assigned module's output so that it could be cross-checked against the other modules, with the evidence for this decision drawn from the test results summarised in Section 4.3. Independently, new knowledge was acquired regarding the practical behaviour of the assigned scheduling/analysis concept beyond its textbook description. If the project were repeated, the assigned module would be redesigned to be validated against a larger and more varied process set from the outset, rather than the single two-process sample used in this iteration.

Lasyasree K: The most difficult engineering problem encountered while working on Proposed System design was ensuring that the module's logic remained consistent with the shared performance formulae used across the rest of the system; this was resolved through direct comparison against hand-calculated values for the sample process set. A key engineering decision made personally concerned how to structure and verify the assigned module's output so that it could be cross-checked against the other modules, with the evidence for this decision drawn from the test results summarised in Section 4.3. Independently, new knowledge was acquired regarding the practical behaviour of the assigned scheduling/analysis concept beyond its textbook description. If the project were repeated, the assigned module would be redesigned to be validated against a larger and more varied process set from the outset, rather than the single two-process sample used in this iteration.

REFERENCES

1. Pakzad, A.E., Africa, A.D.M., Agulto, R.P., Dulay, A.E. and Velasco, N.O.M. (2024) ‘Real-Time Process Monitor: A Linux System Process Visualization and Management Tool’, in 2024 IEEE 16th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM). IEEE. doi: 10.1109/HNICEM64917.2024.11258652.
2. Vivas, A., Tchernykh, A. and Castro, H. (2024) ‘Trends, Approaches, and Gaps in Scientific Workflow Scheduling: A Systematic Review’, IEEE Access, 12, pp. 182203–182231. doi: 10.1109/ACCESS.2024.3509218.
3. Reddy, N.A. and Gokulnath, B.V. (2024) ‘Design of an Improved Method for Task Scheduling Using Proximal Policy Optimization and Graph Neural Networks’, IEEE Access, 12, pp. 1–14. doi: 10.1109/ACCESS.2024.3500046.
4. Mangalampalli, S.S., Karri, G.R., Mohanty, S.N., Ali, S., Ijaz Khan, M., Abdullaev, S. and Alqahtani, S.A. (2024) ‘Multi-Objective Prioritized Task Scheduler Using Improved Asynchronous Advantage Actor Critic (A3C) Algorithm in Multi Cloud Environment’, IEEE Access, 12, pp. 11354–11377. doi: 10.1109/ACCESS.2024.3355092.
5. Wuyts, B., vanden Broucke, S.K.L.M. and De Weerd, J. (2024) ‘SuTraN: An Encoder-Decoder Transformer for Full-Context-Aware Suffix Prediction of Business Processes’, in 2024 6th International Conference on Process Mining (ICPM), pp. 17–24. IEEE. doi: 10.1109/ICPM63005.2024.10680671.
6. Kaundal, R., Soni, S.K. and Rajguru, S. (2024) ‘SCADA-Enhanced Real-Time OEE Visualization Driving Industry 4.0 Advancements’, in 2024 International Conference on Smart Systems for Applications in Electrical Sciences (ICSSES), pp. 1–6. IEEE. doi: 10.1109/ICSSES62373.2024.10561431.
7. Cheng, L., He, H., Gu, Y., Liu, Q., Zhao, Z. and Fang, F. (2024) ‘MARS: Multi-Agent Deep Reinforcement Learning for Real-Time Workflow Scheduling in Hybrid Clouds with Privacy Protection’, in 2024 IEEE 30th International Conference on Parallel and Distributed Systems (ICPADS), pp. 657–666. IEEE.
8. Roider, J., Nguyen, A., Zanca, D. and Eskofier, B.M. (2024) ‘Assessing the Performance of

- Remaining Time Prediction Methods for Business Processes', IEEE Access, 12, pp. 130583–130601. doi: 10.1109/ACCESS.2024.3458405.
9. Subalatha, M., Indu, V.T., Al-Safarini, M.Y., Balakrishna, B., Muthuchelvi, P. and Abinash, M. (2025) 'Adaptive Scheduling and Managing Resources in Changing Industrial Settings: Deep Reinforcement Learning for the Internet of Things', in 2025 IEEE 2nd International Conference on Information Technology, Electronics and Intelligent Communication Systems (ICITEICS). IEEE. doi: 10.1109/ICITEICS64870.2025.11341125.
 10. Li, Y., He, Y., Lin, J., Xu, Z. and Zhang, S. (2025) 'A Reinforcement Learning-Based Population Hyper-Heuristic for Energy-Efficient Cloud Workflow Scheduling Problem', IEEE Transactions on Services Computing, 18(5), pp. 2545–2558. doi: 10.1109/TSC.2025.3589126.
 11. Chen, B., Zhao, S., Zhang, Q., Tang, C. and Lin, L. (2025) 'Dual-View Deep Learning Approach for Predictive Business Process Monitoring', IEEE Transactions on Services Computing, 18(3), pp. 1368–1380. doi: 10.1109/TSC.2025.3562344.
 12. Iman, A.N., Kamal, I.M., Kim, D. and Bae, H. (2025) 'Continual Learning Approach With Adaptive Memory Mechanism for Predictive Process Monitoring in Dynamic Business Environments', IEEE Access, 13, pp. 206767–206783. doi: 10.1109/ACCESS.2025.3636194.
 13. Zheng, B., Yang, S.S., Bai, Y., Tan, M.K. et al. (2024) 'Development of Digital Twin Technology Based Intelligent System for the Manufacturing Processing Line', in 2024 IEEE 22nd Student Conference on Research and Development (SCOREd). IEEE. doi: 10.1109/SCOREd64708.2024.10872652.
 14. Chafi, S.-E., Balboul, Y., Fattah, M., Mazer, S. and El Bekkali, M. (2024) 'Novel PSO-Based Algorithm for Workflow Time and Energy Optimization in a Heterogeneous Fog Computing Environment', IEEE Access, 12, pp. 41517–41530. doi: 10.1109/ACCESS.2024.3377236.
 15. Varshney, S. and Srivastava, G.M.S. (2024) 'Efficient Workflow Scheduling in Fog-Cloud Environments Using Ant Colony Optimization', in 2024 Second International Conference on Data Science and Information System (ICDSIS), pp. 1–5. IEEE. doi: 10.1109/ICDSIS61070.2024.10594408

APPENDICES

Appendix A – Detailed Calculations: See Section 4.4, Tables 11 and 12, for the detailed CT/WT/TAT/RT calculations underlying the scheduling and performance-tracking results.

Appendix B – Engineering Drawings / Schematics: Not applicable – this is a software-only system; the module architecture is described in Section 3.5.

Appendix C – Source Code / Code Repository Information: The complete C source code for all five modules is maintained in the team's project repository/WSL project folder. Only representative outputs are included in this report (Figures 1–7).

Appendix D – Datasheets: Not applicable.

Appendix E – Experimental / Raw Data: See Figures 2–7 for raw console output used as the basis of the results in Section 4.4.

Appendix F – Ethics / Institutional Approval: Not applicable – the project does not involve human/animal subjects or sensitive data.

Appendix G – Project Meeting / Progress Records: Maintained separately by the team as informal meeting notes during the staged development described in Section 4.1.

Appendix H – User/Industry Feedback: Not applicable at this stage of the project.

Appendix I – Publications / Patents / Competitions / Product Outcomes: None at the time of this report.

Appendix J – Individual Contribution Evidence: See the Individual Contribution Statement and Table 14 (Section 4.8).

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)